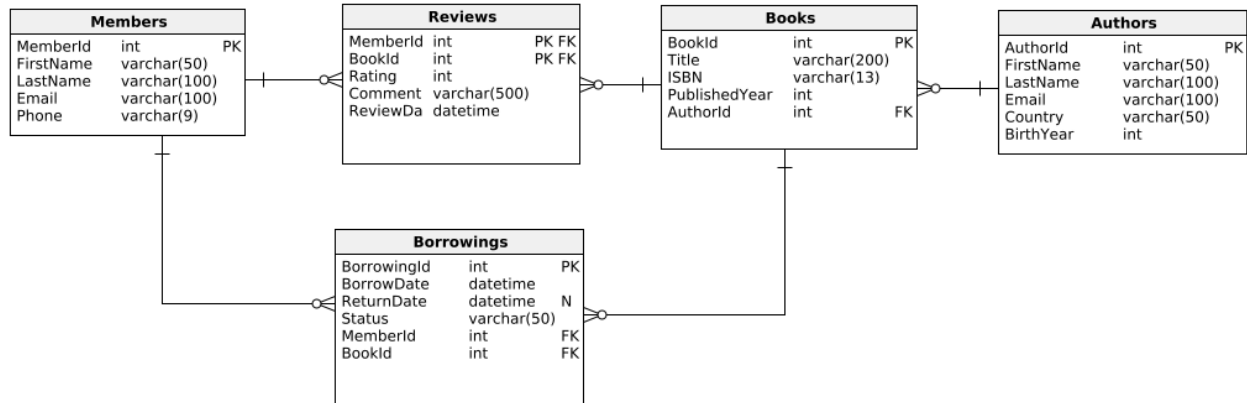


Test EFCore CodeFirst Gr. B

Time: 1h25m

Based on the attached ERD diagram, create a WebApi application and solve the tasks below.



Task1.

Create an Entity Framework Core context using the Code First approach.

- Place models in the **Entities** directory
- Place the database context in the **Data** directory
- Place migrations in the **Migrations** directory

Task2.

Prepare an endpoint at **/api/members** responding to HTTP GET requests. The endpoint should return data about all members present in the database along with their borrowings. Each borrowing should include the book details and the book's author.

Additionally, the endpoint should support an optional parameter: email - filtering the result collection based on the member's email address.

Example requests:

- **GET** /api/members,
- **GET** /api/members?email=anna,
- **GET** /api/members?email=anna@email.com

The required response format can be found in the file GET.json.

Task3.

Prepare an endpoint at **/api/borrowings/{id}/return** responding to HTTP PUT requests. The endpoint should allow marking an existing borrowing as returned by setting its ReturnDate and changing its Status to "Returned".

The operation should be aborted if the borrowing with the given ID does not exist in the database, if the borrowing has already been returned (Status is "Returned"), or if the provided return date is earlier than the borrowing's BorrowDate.

The operation must be performed within a single **transaction**.

The required format of submitted data can be found in the file PUT.json.

Remember about:

- setting up a transaction if necessary;
- validating data sent to the server for POST and PUT endpoints;
- returning appropriate HTTP status codes with messages;
- proper naming conventions for all objects;
- submitting all key files to the repository (including the Migrations folder).

Scoring

For the test you can receive a total of 30 points.

- 10 points for each endpoint,
- 10 points for a correctly created and configured database context.

Deductions:

- incorrect naming conventions, unreadable variable names, or messy code up to -4 pts;
- incorrect HTTP response codes up to -5 pts;
- not separating database communication into a separate layer up to -10 pts;
- missing dependency injection up to -5 pts;
- missing .gitignore or unnecessary files (bin, obj) -50%;
- writing database communication synchronously up to -5 pts;
- missing Migrations folder -5 pts.

Notes

- a program that does not compile - score 0;
- a program not in a GitHub repository - score 0;
- a program that is plagiarized - final grade of 2.

Usage of AI tools or solving the exam not independently during the test will result in failing the course.

Detection of sending the contents of the exam, outside of the group will result in failing the course for the sender of the message and the recipients.